**SIMATS ENGINEERING**

**ASSESSMENT TOOL 1**

**COURSE NAME: Artificial Intelligence** **COURSE CODE: CSA1714**

**COURSE OUTCOME COVERED**

**CO5:** Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)

**Assessment Tool Weightage: 50%**

**Duration: 1 hour**

**QUESTIONS: 5** **Total Marks:50**

**Annexure A**

**Design Task**

**Code of Conduct:**

I **Vempati Murale Karrtik (192312567)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

**Vempati Murale Karrtik**

**Signature of the student:**

## 1. Aim

To design a scalable AI-based Network Intrusion Detection System (NIDS) using PySpark RDDs for processing large volumes of network traffic logs in near real time. The framework distributes traffic analysis across a Spark cluster and uses collaborating intelligent agents to identify suspicious patterns, generate security alerts and continuously improve detection rules from newly observed attack signatures.

## 2. Problem Understanding & Requirement Analysis

Modern networks generate large quantities of traffic records from routers, firewalls, servers, endpoints and applications. A centralized intrusion detection system can become a bottleneck when the traffic volume grows rapidly. The proposed framework solves this problem by partitioning network logs and processing them in parallel across Spark worker nodes.

### Key Requirements

- Process high-volume network traffic records efficiently.
- Distribute traffic records across cluster worker nodes.
- Clean and transform raw logs into machine-learning features.
- Detect both abnormal and known suspicious traffic patterns.
- Use intelligent agents to correlate evidence from multiple analysis stages.
- Generate alerts with source, threat type, severity and confidence.
- Update candidate detection rules when new attack signatures are confirmed.
- Support horizontal scalability and fault-tolerant distributed processing.

### Expected Outcome

The expected output is an integrated pipeline in which network traffic is ingested, distributed through PySpark RDDs, transformed into useful features, analyzed for anomalies, evaluated by collaborating agents and converted into a security decision.

| Src IP | Dst IP | Protocol | Dst Port | Packets | Bytes | Duration(s) | Failed Conn. | Label |
|---|---|---|---|---|---|---|---|---|
| 192.168.1.10 | 10.0.0.5 | TCP | 443 | 35 | 18200 | 12 | 0 | Normal |
| 192.168.1.21 | 10.0.0.8 | TCP | 22 | 180 | 95600 | 8 | 5 | Suspicious |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 192.168.1.50 | 10.0.0.20 | TCP | 23 | 420 | 215000 | 15 | 32 | Port Scan |
| 192.168.1.75 | 10.0.0.12 | UDP | 53 | 65 | 31000 | 20 | 1 | Normal |
| 192.168.1.90 | 10.0.0.15 | TCP | 80 | 950 | 1250000 | 6 | 48 | Suspicious |

**NETWORK LOGS → RDD PROCESSING → FEATURE EXTRACTION → ANOMALY DETECTION → AGENT COLLABORATION → THREAT DECISION → ALERT / RULE UPDATE**

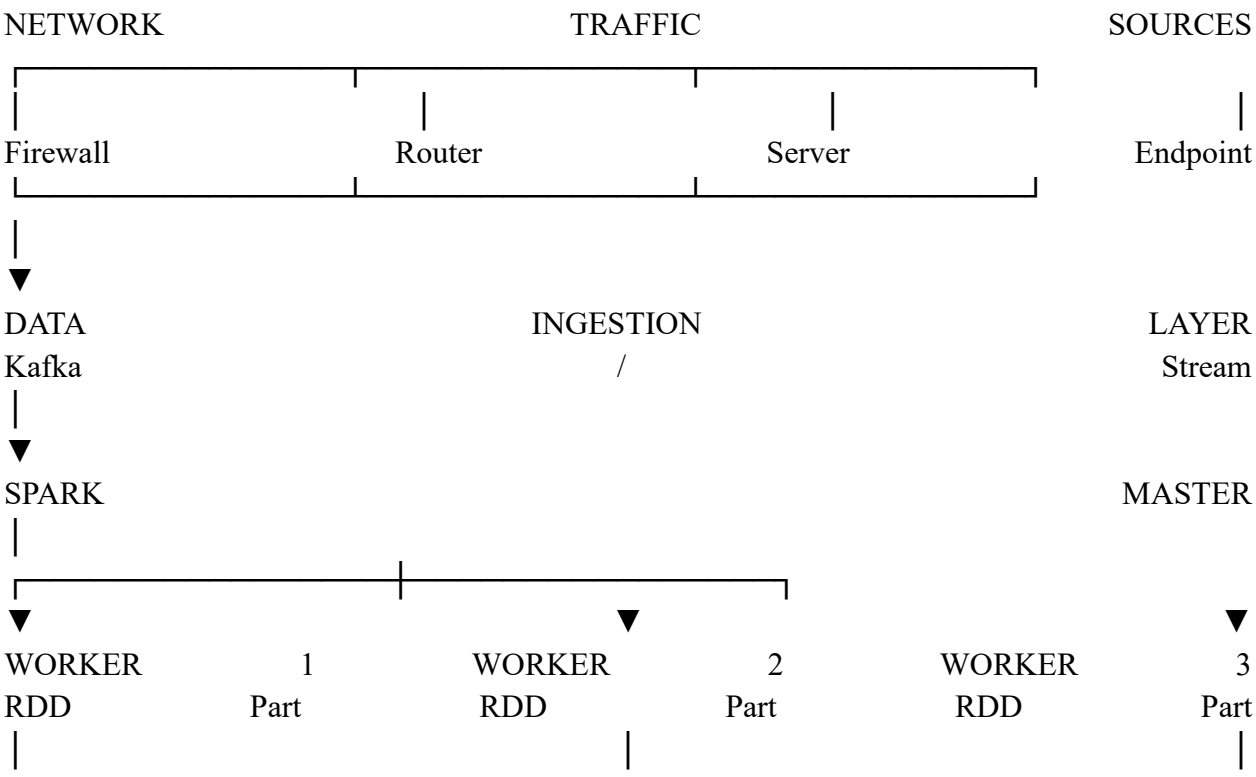
**3. Sample Network Traffic Inputs**

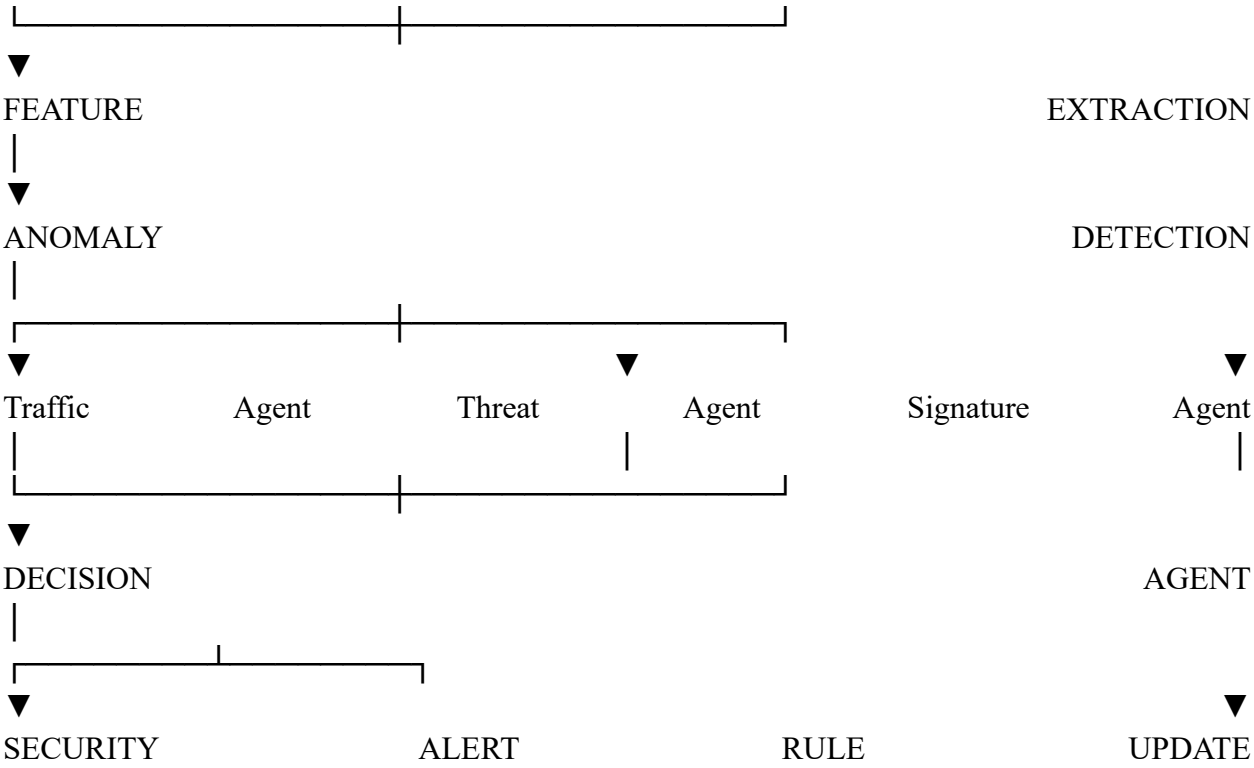
The following sample records are inserted to demonstrate the type of input handled by the proposed system. In an actual deployment, the same fields can be generated continuously by firewall, router, server or IDS logs.

AI-Based Network Intrusion Detection Framework

4. System Architecture Design – 20%

The proposed architecture contains a data-ingestion layer, a distributed Spark-processing layer, an anomaly-detection layer, a multi-agent intelligence layer and a security-response layer.






Major Components

| Component | Function |
|---|---|
| Log Collection | Collects traffic metadata from network devices. |
| Ingestion Layer | Transfers high-volume records into the processing environment. |
| Spark Master | Coordinates distributed computation and cluster resources. |
| Worker Nodes | Process assigned RDD partitions in parallel. |
| RDD Processing | Performs transformations and aggregations on network records. |
| Feature Extraction | Creates features such as packet rate, byte count and failed connections. |
| Anomaly Detector | Calculates whether observed behavior deviates from normal traffic. |

| | |
|---|---|
| Intelligent Agents | Collaborate to monitor, correlate and interpret suspicious events. |
| Decision Agent | Assigns threat severity and selects the response. |
| Rule Manager | Stores validated new attack signatures and detection rules. |

5. PySpark & RDD Operations – 15%

RDDs provide a distributed collection abstraction in Spark. Network records are partitioned across worker nodes so that multiple records can be analyzed concurrently.

Example input creation: logs = sc.textFile("network_logs.csv")

Example transformations and actions:

records = logs.map(parse_log)

valid = records.filter(lambda x: x["src_ip"] != "")

traffic = records.map(lambda x: (x["src_ip"], x["bytes"]))

ip_bytes = traffic.reduceByKey(lambda a, b: a + b)

unique_ips = records.map(lambda x: x["src_ip"]).distinct()

AI-Based Network Intrusion Detection Framework

6. Distributed Anomaly Detection

Anomaly detection can be implemented by applying the same detection function independently to RDD partitions. Each worker calculates local statistics or anomaly scores and Spark subsequently combines the results. This approach avoids moving every raw record to a single central machine.

Example distributed flow:

10 Million Logs → RDD Partitions → Worker 1 / Worker 2 / Worker 3 / … → Local Anomaly Scores → Aggregated Threat Results

Feature Set for Intrusion Detection

| Feature | Purpose |
|---|---|
| Packet Count | Measures connection activity. |
| Byte Count | Identifies unusually high data transfer. |
| Connection Duration | Helps identify abnormal sessions. |

| | |
|---|---|
| Destination Port | Useful for service and port-scan detection. |
| Protocol | Distinguishes TCP, UDP and other traffic. |
| Failed Connections | Useful for brute-force and scanning detection. |
| Traffic Rate | Detects sudden bursts and flooding behavior. |
| Unique Destination Ports | Helps identify port scanning. |

7. Intelligent Agent Integration – 15%

Agent 1 – Traffic Monitoring Agent

Monitors incoming records, computes traffic statistics and identifies sudden changes in connection volume, packet rate or destination-port diversity.

Agent 2 – Anomaly Detection Agent

Receives extracted features, calculates an anomaly score and flags records or groups of records whose behavior significantly differs from the learned normal profile.

Agent 3 – Threat Correlation Agent

Combines alerts from different records and identifies relationships such as repeated failed connections from one source IP or simultaneous access to many destination ports.

Agent 4 – Signature Analysis Agent

Compares suspicious patterns with known attack signatures and identifies recurring patterns that could represent a new attack signature.

Agent 5 – Decision / Response Agent

Combines the evidence produced by the other agents, determines threat severity and triggers an appropriate security response.

Agent Collaboration

Traffic Agent → Anomaly Agent → Correlation Agent → Signature Agent → Decision Agent → Alert / Rule Update

8. Example Suspicious Input

| Source IP | Ports Contacted | Failed Connections | Traffic Pattern | Agent Result |
|---|---|---|---|---|
| 192.168.1.50 | 21,22,23,25,53,80,443 | 32 | Many ports in short time | Possible Port Scan |

| | | | | |
|---|---|---|---|---|
| 192.168.1.90 | 22 | 48 | Repeated failed SSH attempts | Possible Brute Force |

AI-Based Network Intrusion Detection Framework

9. Scalability & Distributed Computing Design – 10%

The system is designed for horizontal scaling. When network traffic increases, additional Spark worker nodes can be added rather than replacing the entire system with a larger single server.

| Design Feature | Scalability Benefit |
|---|---|
| RDD Partitioning | Splits traffic into independently processable partitions. |
| Parallel Workers | Multiple nodes analyze traffic simultaneously. |
| Horizontal Scaling | New workers can be added for higher traffic volume. |
| Fault Tolerance | Lost RDD partitions can be recomputed by Spark. |
| Data Locality | Processing can occur close to distributed data when supported. |
| Aggregation | Local results can be combined efficiently. |

10. Data Flow & Processing Pipeline – 10%

1. Network Traffic Collection
↓
2. Data Ingestion
↓
3. RDD Creation
↓
4. Data Cleaning
↓
5. Feature Extraction
↓
6. Distributed Anomaly Detection
↓
7. Intelligent Agent Collaboration
↓

8. Threat Classification
↓
9. Security Alert
↓
10. Candidate Rule Generation / Validated Rule Update

11. Sample Processing Result

| Input Source | Anomaly Score | Threat | Severity | Action |
|---|---|---|---|---|
| 192.168.1.10 | 0.08 | Normal traffic | LOW | Continue monitoring |
| 192.168.1.21 | 0.71 | Suspicious SSH activity | HIGH | Investigate / restrict |
| 192.168.1.50 | 0.91 | Port scanning | CRITICAL | Block source / investigate |
| 192.168.1.90 | 0.86 | Possible brute force | HIGH | Rate-limit / investigate |

The numerical anomaly scores above are illustrative sample outputs used to demonstrate the intended decision flow; actual scores depend on the trained anomaly-detection model and plant/network data.

12. Security Alert Format

SECURITY ALERT
Source IP : 192.168.1.50
Destination : Multiple Ports
Threat : Possible Port Scanning
Severity : CRITICAL
Anomaly Score : 0.91
Confidence : 91%
Recommended : BLOCK SOURCE / INVESTIGATE

AI-Based Network Intrusion Detection Framework

13. Innovation & Creativity – 10%

Hybrid Detection

The framework combines anomaly-based analysis with signature-based analysis. Anomaly detection can identify previously unseen behavior, while signature analysis provides fast recognition of known attack patterns.

Collaborative Intelligent Agents

Instead of relying on one decision module, specialized agents independently analyze different aspects of traffic and exchange evidence. This improves threat correlation and reduces dependence on a single detection rule.

Continuous Rule Improvement

When a suspicious pattern is repeatedly observed and confirmed by the security decision process, the Signature Agent can create a candidate rule. The candidate is validated by the security team or policy engine before being added to the production rule repository. This controlled approach avoids automatically poisoning the rule base with false positives.

14. End-to-End Example

Consider a source IP that generates hundreds of connection attempts to many ports within a short interval.

| Stage | Processing | Output |
|---|---|---|
| 1. Input | Network log arrives | Source 192.168.1.50 |
| 2. RDD | Record assigned to worker partition | Distributed processing |
| 3. Features | Ports, packets and failures extracted | High port diversity |
| 4. Anomaly | Behavior compared with normal profile | Anomaly score = 0.91 |
| 5. Correlation | Agent groups related events | Port-scan pattern |
| 6. Signature | Compared with known signatures | Possible scan |
| 7. Decision | Severity calculated | CRITICAL |
| 8. Response | Alert generated | Block / investigate |
| 9. Learning | New confirmed pattern considered | Candidate rule update |

15. Advantages of Proposed Framework

Handles large traffic volumes through distributed RDD processing.

Supports parallel anomaly analysis across multiple worker nodes.

Combines multiple specialized intelligent agents.

Provides structured and explainable security alerts.

Can detect both known and potentially unknown attack behavior.

Supports controlled continuous improvement of detection rules.

Can scale by adding additional Spark worker nodes.

16. Limitations and Design Considerations

RDD-based processing is primarily batch-oriented; a production real-time implementation should pair Spark processing with an appropriate streaming/ingestion layer.

Anomaly thresholds must be tuned to the network environment to reduce false positives.

Automatically generated rules require validation before production deployment.

Security logs may contain sensitive information and therefore require appropriate access control and retention policies.

**18. Conclusion**

The proposed AI-Based Network Intrusion Detection Framework provides a scalable architecture for processing large volumes of network traffic using PySpark RDDs. By partitioning network records across Spark worker nodes, feature extraction and anomaly analysis can be performed in parallel.

The intelligent-agent layer strengthens the framework by assigning specialized responsibilities to traffic monitoring, anomaly detection, threat correlation, signature analysis and decision-making agents. Their collaboration converts individual suspicious observations into a more reliable security decision.

The framework also supports continuous improvement through controlled candidate-rule generation when new attack signatures are discovered. Overall, the design combines distributed computing, AI-based anomaly detection and collaborative intelligent agents into a scalable and adaptive intrusion detection solution.

**19. Future Scope**

Integrate Spark Structured Streaming or another streaming layer for production-grade continuous processing.

Train anomaly-detection models using real network datasets such as enterprise traffic captures.

Add deep-learning models for complex sequential attack patterns.

Introduce threat-intelligence feeds for external indicator enrichment.

Use automated feedback from confirmed incidents to improve detection thresholds.

Deploy the system on a multi-node cloud or on-premise Spark cluster.

**20. Software / Technologies**

Python, PySpark, Apache Spark, RDD, machine-learning/anomaly-detection algorithms, Kafka or equivalent ingestion technology, and a security-rule/alert management layer.